

Задача 7. “Замены и перестановки”

Алгоритм развертывания генерирует ключевую информацию порциями по 256 бит.

Первые 256 бит будут получены как результат зашифрования нулевой последовательности алгоритмом Skip-jack на секретном 80-битном ключе K_{root} и с известной 64-битной синхропосылкой

$$IV = \{0x43, 0x72, 0x79, 0x70, 0x74, 0x46, 0x6f, 0x78\}$$

в режиме гаммирования с обратной связью по шифртексту ($K_{root} \in \mathbb{F}_2^{80}$, $IV \in \mathbb{F}_2^{64}$):

$$K_0 = \text{Encrypt}_{cfb}(K_{root}, IV, [0x00] \cdot 32),$$

где $K_0 \in \mathbb{F}_2^{256}$.

Пусть $\text{Enc}(P)$ - функция шифрования 64-битного открытого текста P с использованием ключа K_{root} .

Допустим $K_0 = (K_0[0], K_0[1], K_0[2], K_0[3]) \in \mathbb{F}_2^{256}$, где $K_0[j] \in \mathbb{F}_2^{64}$ для $j = 0, 1, 2, 3$. По режиму гаммирования с обратной связью:

$$\begin{aligned} K_0[0] &= \text{Enc}(IV) \oplus ([0x00] \cdot 8) = \text{Enc}(IV), \\ K_0[1] &= \text{Enc}(K_0[0]) \oplus ([0x00] \cdot 8) = \text{Enc}(K_0[0]), \\ K_0[2] &= \text{Enc}(K_0[1]) \oplus ([0x00] \cdot 8) = \text{Enc}(K_0[1]), \\ K_0[3] &= \text{Enc}(K_0[2]) \oplus ([0x00] \cdot 8) = \text{Enc}(K_0[2]). \end{aligned}$$

Следующие 256 бит будут получены как результат зашифрования ранее выработанной порции ключевой информации:

$$K_1 = \text{Encrypt}_{cfb}(K_{root}, IV, K_0).$$

Допустим $K_1 = (K_1[0], K_1[1], K_1[2], K_1[3]) \in \mathbb{F}_2^{256}$, где $K_1[j] \in \mathbb{F}_2^{64}$ для $j = 0, 1, 2, 3$. Аналогично, по режиму гаммирования с обратной связью:

$$\begin{aligned} K_1[0] &= \text{Enc}(IV) \oplus K_0[0], \\ K_1[1] &= \text{Enc}(K_0[0]) \oplus K_0[1], \\ K_1[2] &= \text{Enc}(K_0[1]) \oplus K_0[2], \\ K_1[3] &= \text{Enc}(K_0[2]) \oplus K_0[3]. \end{aligned}$$

Выработка всех остальных порций выполняется аналогично первой:

$$K_i = \text{Encrypt}_{cfb}(K_{root}, IV, K_{i-1}),$$

где $i \in \mathbb{N}$, $K_i \in \mathbb{F}_2^{256}$. Снова, допустим $K_i = (K_i[0], K_i[1], K_i[2], K_i[3]) \in \mathbb{F}_2^{256}$, где $K_i[j] \in \mathbb{F}_2^{64}$ для $j = 0, 1, 2, 3$. Аналогично, по режиму гаммирования с обратной связью:

$$\begin{aligned} K_i[0] &= \text{Enc}(IV) \oplus K_{i-1}[0], \\ K_i[1] &= \text{Enc}(K_{i-1}[0]) \oplus K_{i-1}[1], \\ K_i[2] &= \text{Enc}(K_{i-1}[1]) \oplus K_{i-1}[2], \\ K_i[3] &= \text{Enc}(K_{i-1}[2]) \oplus K_{i-1}[3]. \end{aligned}$$

Гамма формируется на основе полученных значений K_i и K_0 . Первые 4 бита гаммы будут получены из K_0 по следующему правилу:

$$\gamma[j] = \begin{cases} 0, & \text{если } K_0[j] = 0 \\ 1, & \text{другие} \end{cases},$$

где $K_0[j]$ - j -ый 64-битный блок K_0 , $\gamma[j] \in \mathbb{F}_2$, $j = 0, 1, 2, 3$.

Следующие биты формируются аналогично на основе новых порций ключевой информации, т.е. гамма формируется так:

$$\gamma[j] = \begin{cases} 0, & \text{если } K_{j/4}[j\%4] = 0, \\ 1, & \text{другие} \end{cases},$$

где $a\%b$ - остаток от деления числа a на b , $j \in \mathbb{N}_0$.

1 skipjack.py

Сначала нам нужен файл для шифрования по алгоритму Skipjack и для режима гаммирования с обратной связью.

```
# skipjack.py
class SkipJack:
    def __init__(self):
        # F is an 8-bit S-box
        self.F = []
        self.defineF()
        # w1, w2, w3, w4 are 16-bit integers
        self.w1 = 0
        self.w2 = 0
        self.w3 = 0
        self.w4 = 0

        # plaintext is a 64-bit integer
        # key is a list of 10-bytes
    def encrypt(self, plaintext, key):
        self.splitWord(plaintext)

        for round in range(1, 33):
            if (1 <= round <= 8) or (17 <= round <= 24):
                self.A(round, key)
                self.printStatus(round)
            if (9 <= round <= 16) or (25 <= round <= 32):
                self.B(round, key)
                self.printStatus(round)

        return self.appendWords()

        # ciphertext is a 64-bit integer
        # key is a list of 10-bytes
    def decrypt(self, ciphertext, key):
        self.splitWord(ciphertext)

        for round in reversed(range(1, 33)):
            if (25 <= round <= 32) or (9 <= round <= 16):
                self.Binv(round, key)
            if (17 <= round <= 24) or (1 <= round <= 8):
                self.Ainv(round, key)

        return self.appendWords()

    def A(self, round, key):
        c1 = self.w1
        c2 = self.w2
        c3 = self.w3
        self.w1 = self.G(round, key, c1) ^ self.w4 ^ round
        self.w2 = self.G(round, key, c1)
        self.w3 = c2
        self.w4 = c3

    def Ainv(self, round, key):
        c1 = self.w1
        c2 = self.w2
        self.w1 = self.Ginv(round, key, c2)
        self.w2 = self.w3
        self.w3 = self.w4
        self.w4 = c1 ^ c2 ^ round
```

```

def B(self, round, key):
    c1 = self.w1
    c2 = self.w2
    c3 = self.w3
    self.w1 = self.w4
    self.w2 = self.G(round, key, c1)
    self.w3 = c1 ^ c2 ^ round
    self.w4 = c3

def Binv(self, round, key):
    c1 = self.w1
    self.w1 = self.Ginv(round, key, self.w2)
    self.w2 = self.Ginv(round, key, self.w2) ^ self.w3 ^ round
    self.w3 = self.w4
    self.w4 = c1

# w is a 16-bit integer
def G(self, round, key, w):
    g = [0] * 6
    g[0] = (w >> 8) & 0xff
    g[1] = w & 0xff
    j = (4 * (round - 1)) % 10

    for i in range(2, 6): # gives i = 2,3,4,5
        g[i] = self.F[g[i - 1] ^ key[j]] ^ g[i - 2]
        j = (j + 1) % 10

    return (g[4] << 8) | g[5]

# w is a 16-bit integer
def Ginv(self, round, key, w):
    g = [0] * 6
    g[4] = (w >> 8) & 0xff
    g[5] = w & 0xff
    j = (4 * (round - 1) + 3) % 10

    for i in reversed(range(4)): # gives i=3,2,1,0
        g[i] = self.F[g[i + 1] ^ key[j]] ^ g[i + 2]
        j = (j - 1) % 10

    return (g[0] << 8) | g[1]

# append the four 16-bit words w1,w2,w3,w4 to return a 64-bit word
def appendWords(self):
    x1 = self.w1 << 3 * 16
    x2 = self.w2 << 2 * 16
    x3 = self.w3 << 1 * 16
    x4 = self.w4
    return x1 | x2 | x3 | x4

# w is a 64-bit word. This function splits w into
# four 16-bit words which are stored in w1, w2, w3, w4
def splitWord(self, w):
    self.w1 = (w >> (16 * 3)) & 0xffff
    self.w2 = (w >> (16 * 2)) & 0xffff
    self.w3 = (w >> (16 * 1)) & 0xffff
    self.w4 = w & 0xffff

# print the round number and the current values of w1,w2,w3,w4
def printStatus(self, round):

```

```

w = self.appendWords()
# print("round=" + str(round) + " " + hex(w))

def defineF(self):
    self.F = [0xa3, 0xd7, 0x09, 0x83, 0xf8, 0x48, 0xf6, 0xf4,
               0xb3, 0x21, 0x15, 0x78, 0x99, 0xb1, 0xaf, 0xf9,
               0xe7, 0x2d, 0x4d, 0x8a, 0xce, 0x4c, 0xca, 0x2e,
               0x52, 0x95, 0xd9, 0x1e, 0x4e, 0x38, 0x44, 0x28,
               0x0a, 0xdf, 0x02, 0xa0, 0x17, 0xf1, 0x60, 0x68,
               0x12, 0xb7, 0x7a, 0xc3, 0xc9, 0xfa, 0x3d, 0x53,
               0x96, 0x84, 0x6b, 0xba, 0xf2, 0x63, 0x9a, 0x19,
               0x7c, 0xae, 0xe5, 0xf5, 0xf7, 0x16, 0x6a, 0xa2,
               0x39, 0xb6, 0x7b, 0x0f, 0xc1, 0x93, 0x81, 0x1b,
               0xee, 0xb4, 0x1a, 0xea, 0xd0, 0x91, 0x2f, 0xb8,
               0x55, 0xb9, 0xda, 0x85, 0x3f, 0x41, 0xbf, 0xe0,
               0x5a, 0x58, 0x80, 0x5f, 0x66, 0x0b, 0xd8, 0x90,
               0x35, 0xd5, 0xc0, 0xa7, 0x33, 0x06, 0x65, 0x69,
               0x45, 0x00, 0x94, 0x56, 0x6d, 0x98, 0x9b, 0x76,
               0x97, 0xfc, 0xb2, 0xc2, 0xb0, 0xfe, 0xdb, 0x20,
               0xe1, 0xeb, 0xd6, 0xe4, 0xdd, 0x47, 0x4a, 0x1d,
               0x42, 0xed, 0x9e, 0x6e, 0x49, 0x3c, 0xcd, 0x43,
               0x27, 0xd2, 0x07, 0xd4, 0xde, 0xc7, 0x67, 0x18,
               0x89, 0xcb, 0x30, 0x1f, 0x8d, 0xc6, 0x8f, 0xaa,
               0xc8, 0x74, 0xdc, 0xc9, 0x5d, 0x5c, 0x31, 0xa4,
               0x70, 0x88, 0x61, 0x2c, 0x9f, 0x0d, 0x2b, 0x87,
               0x50, 0x82, 0x54, 0x64, 0x26, 0x7d, 0x03, 0x40,
               0x34, 0x4b, 0x1c, 0x73, 0xd1, 0xc4, 0xfd, 0x3b,
               0xcc, 0xfb, 0x7f, 0xab, 0xe6, 0x3e, 0x5b, 0xa5,
               0xad, 0x04, 0x23, 0x9c, 0x14, 0x51, 0x22, 0xf0,
               0x29, 0x79, 0x71, 0x7e, 0xff, 0x8c, 0x0e, 0xe2,
               0x0c, 0xef, 0xbc, 0x72, 0x75, 0x6f, 0x37, 0xa1,
               0xec, 0xd3, 0x8e, 0x62, 0x8b, 0x86, 0x10, 0xe8,
               0x08, 0x77, 0x11, 0xbe, 0x92, 0x4f, 0x24, 0xc5,
               0x32, 0x36, 0x9d, 0xcf, 0xf3, 0xa6, 0xbb, 0xac,
               0x5e, 0x6c, 0xa9, 0x13, 0x57, 0x25, 0xb5, 0xe3,
               0xbd, 0xa8, 0x3a, 0x01, 0x05, 0x59, 0x2a, 0x46]

def encrypt_string(self, input_string, key):
    # Check if the input string length is 8
    if len(input_string) != 8:
        print("Input string must be 8 characters long.")
        return None

    # Convert the input string into a numerical representation
    numerical_input = 0
    for char in input_string:
        numerical_input <= 8 # Shift left by 8 bits
        numerical_input |= ord(char) # Add ASCII value of the character

    # Encrypt the numerical input using SkipJack algorithm
    ciphertext = self.encrypt(numerical_input, key)

    return ciphertext

def xor(a, b):
    return [x^y for x, y in zip(a, b)]

def encrypt_cfb(Kroot: list[int], IV: list[int], Kprev: list[int]):
    K = [iv for iv in IV]

```

```

for i in range(0, 32, 4):
    sj = SkipJack()
    Kj = int.from_bytes(bytes(K[i:i+4]), 'big')
    Kj = sj.encrypt(Kj, Kroot)
    Kj = list(Kj.to_bytes(8, 'big'))
    Kj = xor(Kj, Kprev[i:i+4])
    K.extend(Kj)
return K[8:]

```

2 Поиск закона при генерации подключа find_rules.py

```

# find_rules.py
import random
from skipjack import *

Kroot = list(random.randbytes(10))
IV = [0x43, 0x72, 0x79, 0x70, 0x74, 0x46, 0x6f, 0x78]
Kprev = [0] * 32

for _ in range(16):
    # Encrypt with CFB
    Kprev = encrypt_cfb(Kroot, IV, Kprev)
    # Split K_i into 4 chunks
    s = [bytes(Kprev[i:i+8]).hex() for i in range(0, len(Kprev), 8)]
    # Print K_i as 4 chunks
    print(" ".join(s))

```

Вывод имеет вид:

```

...
0d16dceddfc805c1 47122ce9d3b7983a 02fd48e270666df7 28a8d35b20fbd167
0000000000000000 c4a1be9a50040a49 a266cc2ec5ada370 59c24fdb72f24d49
0d16dceddfc805c1 83b3927383b39273 1374429b74bf2dc5 b2e5c38addfd9bbc
0000000000000000 0000000000000000 90c7d0e8f70cbfb6 b4975af551b93659
0d16dceddfc805c1 47122ce9d3b7983a 923a980a876ad241 f786cb6cd09fa275
0000000000000000 c4a1be9a50040a49 32a11cc632a11cc6 a16a8af18673e3e8
0d16dceddfc805c1 83b3927383b39273 83b3927383b39273 1078044437616d5d
0000000000000000 0000000000000000 0000000000000000 93cb9637b4d2ff2e
0d16dceddfc805c1 47122ce9d3b7983a 02fd48e270666df7 bb63456c94292e49
0000000000000000 c4a1be9a50040a49 a266cc2ec5ada370 ca09d9ecc620b267
0d16dceddfc805c1 83b3927383b39273 1374429b74bf2dc5 212e55bd692f6492
0000000000000000 0000000000000000 90c7d0e8f70cbfb6 275cccc2e56bc977
0d16dceddfc805c1 47122ce9d3b7983a 923a980a876ad241 644d5d5b644d5d5b
0000000000000000 c4a1be9a50040a49 32a11cc632a11cc6 32a11cc632a11cc6
0d16dceddfc805c1 83b3927383b39273 83b3927383b39273 83b3927383b39273
0000000000000000 0000000000000000 0000000000000000 0000000000000000
...

```

Используем некоторые случайные ключ K_{root} , видим, что вывод имеет общий вид

```

...
-----
0000000000000000 -----
-----
0000000000000000 0000000000000000 -----
-----
0000000000000000 -----
-----
0000000000000000 0000000000000000 0000000000000000 -----
-----
0000000000000000 -----
-----

```

```

-----
0000000000000000 0000000000000000 -----
-----
0000000000000000 -----
-----
0000000000000000 0000000000000000 0000000000000000 0000000000000000
...

```

где ----- обозначает значение, которое не равно нулю.

Отсюда можем сказать, что последовательность $\gamma[j]$ не зависит от исходный ключ K_{root} . Поэтому можем использовать любой ключ K_{root} , и сгенерируем последовательность γ .

3 Главный файл solve.py

Главный файл для восстановления гаммирования и открытого текста.

```

# solve.py
import random
from skipjack import *

with open("ct.txt", encoding="cp1251") as f:
    data = f.read()
    dd = [ord(d.encode("cp1251")) for d in data]

Kroot = list(random.randbytes(10))
IV = [0x43, 0x72, 0x79, 0x70, 0x74, 0x46, 0x6f, 0x78]
Kprev = [0] * 32
flag = ""
# Generate K_i with random K_{root}
for _ in range(len(data) * 2):
    # Calculate K_i from K_{i-1} with CFB
    Kprev = encrypt_cfb(Kroot, IV, Kprev)

    F = bytes(Kprev).hex()
    for i in range(0, len(F), 16):
        if F[i:i+16] == "0" * 16: # Check K_i[j] == 0 for j = 0, 1, 2, 3
            flag += "0"
        else:
            flag += "1"

gamma = [int(flag[i:i+8], 2) for i in range(0, len(flag), 8)]

pt = [g ^ d for g, d in zip(gamma, dd)]
print(bytes(pt))

```

4 Анализ результата

EVALUATION OF THE SITUATION UNTIL NOW IT HAD TO BE EXPECTED THAT THE ENEMY WOULD TRY TO CROSS THE DANUBE WITH THE TROOPS ASSEMBLING AT VIDIN LOMPALANKA AND NEAR DRUTSCHUK WITH THE GOAL TO CUT OFF THE RAILWAYS BETWEEN ORSOVA AND CRAIOVA AND TO PUSH FORWARD TO BUCHAREST SINCE NOVEMBER 1 1918 IT APPEARS THAT THE SERBIAN ARMIES TOGETHER WITH THREE FRENCH DIVISIONS ARE ENGAGED IN AN ADVANCE TOWARD BELGRADE SEMENDRIA AND THE INTENDED ATTACK AT VIDI AND LOM PALANKA SEEMS TO HAVE BEEN ABANDONED IT HAS TO BE EXPECTED THE DEPLOYMENT OF STRONGER FORCES AT THE DANUBE SOUTH OF SVISTOVRUSTSCHUK SPECIALLY AFTER THE PEACE AGREEMENT OF TURKEY HENCE IT IS MOST LIKELY THAT THE SERBIAN ARMIES REINFORCED BY THE FRENCH INTEND TO CROSS THE DANUBE NEAR BELGRADE SEMENDRIA AND THE INTO SOUTHERN REPEAT SOUTHERN HUNGARY WHILE THE TASK OF THE FRENCH ARMY MARCHING SOUTH OF SVISTOV RUTSCHUK REMAINS THE OFFENSIVE TOWARD

BUCHAREST IN CONJUNCTION WITH THIS OPERATION IT CANNOT BE RULED OUT THAT THE ROMANIAN FORCES COMING FROM MOLDAVIA OVER THE PASSES OF TOELGYESGIMES AND OITOS WILL INVADE TRANSYLVANIA THUS THE LINES OF COMMUNICATIONS IN THE REAR OF THE OCCUPATION ARMY WHICH HAVE UP TO NOW AS A RESULT OF IS THREATENED WITH ATTACK AND THE FURTHER OCCUPATION OF THE WALL ACHIAAS LAID DOWN IN ORDER FROM OKHEAD QUARTERS 2RMNR11161 WITHOUT DOUBT AND WITH REGARD TO THE AMMUNITION FOOD AND THE COAL STOCK SIT IS UNFEASIBLE IF THE GENERAL ARM ISTICE DOES NOT BECOME EFFECTIVE IN THE FORESEEABLE FUTURE IT IS SUGGESTED THAT THE OCCUPATION ARMY BE WITHDRAWN ATONCE FROM ROMANIA AND TOGETHER WITH THE GERMAN UNITS OF THE FIRST ARMY TO START THE MARCH TOUPPERSILES IA THROUGH HUNGARY APPROVAL IS REQUESTED SIGNED KMIAGROP

В конце текста видим SIGNED KMIAGROP, значит фамилия автора является **KMIAGROP**.
Ответ: **KMIAGROP**.